



UNIVERSIDADE FEDERAL DO AMAPÁ
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Disciplina: Engenharia de Software I
Professor: Dr. Adolfo Colares

Daniela Marques Haber Sepeda
Elton Braian Mendes da Silva
Geovanni Rodrigues da Silva
Maria Fernanda Fernandes de Souza Gonçalves
Vitória Christina Ataíde Rodrigues

DOCUMENTO DE ARQUITETURA E DESIGN DE SOFTWARE

Plataforma de Anúncios de Estágios Acadêmicos – Portal do Saber

Macapá
2026

SUMÁRIO

1. Introdução	3
1.1. Objetivo do documento	3
1.2. Escopo	3
1.3. Definições, acrônimos e abreviações.....	3
1.4. Referências	3
2. Representação Arquitetural	4
2.1. Visão geral	4
2.2. Padrão arquitetural adotado	4
2.3. Justificativa da escolha	4
3. Diagrama de Arquitetura	6
3.1. Diagrama de componentes em camadas	6
3.2. Descrição dos componentes.....	6
3.3. Fluxo de uma requisição típica	7
4. Decisões de Design e Princípios SOLID	8
4.1. DIP — Princípio da Inversão de Dependência	8
4.2. OCP — Princípio do Aberto/Fechado	8
4.3. LSP — Princípio da Substituição de Liskov	8
4.4. SRP — Princípio da Responsabilidade Única	8
5. Padrões de Projeto (Design Patterns)	9
5.1. Repository (Repositório)	9
5.2. Provider / Factory (Fábrica)	9
5.3. Strategy (Estratégia)	9
5.4. Decorator (Decorador).....	9
6. Tecnologias Utilizadas e Justificativas.....	11
7. Atributos de Qualidade Suportados pela Arquitetura.....	11
8. Considerações Finais	12

1. INTRODUÇÃO

1.1. Objetivo do documento

Este documento descreve a **arquitetura de software** e as **principais decisões de design** adotadas na construção da Plataforma de Anúncios de Estágios Acadêmicos. Seu propósito é registrar, de forma rastreável, o padrão arquitetural empregado, a organização dos componentes do sistema, os princípios de projeto orientado a objetos aplicados (SOLID) e os padrões de projeto (design patterns) utilizados, bem como justificar as tecnologias escolhidas. As descrições aqui apresentadas correspondem fielmente ao código-fonte presente no repositório do projeto na data desta entrega.

1.2. Escopo

O escopo do documento abrange a camada de **back-end e aplicação web** implementada em Python com o framework Django, contida no diretório `backend/`. São detalhadas as camadas de apresentação, interface HTTP, serviços de aplicação, domínio e persistência. Aspectos de requisitos, modelagem UML e testes são tratados em documentos próprios e aqui referenciados apenas quando pertinentes à arquitetura.

1.3. Definições, acrônimos e abreviações

Termo	Definição
API	Application Programming Interface — interface de comunicação HTTP/JSON exposta pelo back-end.
DTO	Data Transfer Object — objeto usado para transportar dados entre camadas sem expor a estrutura interna.
ORM	Object-Relational Mapping — mapeamento objeto-relacional provido pelo Django.
SOLID	Conjunto de cinco princípios de projeto orientado a objetos (SRP, OCP, LSP, ISP, DIP).
MVT	Model-View-Template — variação do padrão MVC adotada pelo Django.
Repositório	Componente que abstrai o acesso e a persistência dos dados do domínio.

1.4. Referências

- Documento de Requisitos de Software — `docs/entregaveis-finais/documento-de-requisitos.md`
- Modelagem do Sistema (Diagrama de Classes) — `docs/modelagem/`

- Documento de Gestão de Processo — docs/entregaveis-finais/gestao-de-processo.md
- Código-fonte da aplicação — backend/

2. REPRESENTAÇÃO ARQUITETURAL

2.1. Visão geral

A aplicação adota uma **arquitetura em camadas (layered architecture)** com forte separação de responsabilidades. O núcleo de regras de negócio é mantido **independente do framework** e da tecnologia de persistência: as regras residem em funções de serviço puras que operam sobre entidades de domínio e dialogam com a base de dados exclusivamente por meio de uma **abstração de repositório**. Essa organização é reforçada por uma decisão arquitetural central do projeto: a **persistência é intercambiável** — a mesma lógica de negócio funciona tanto sobre um repositório em memória quanto sobre um repositório baseado no ORM do Django, selecionado em tempo de configuração.

2.2. Padrão arquitetural adotado

O padrão adotado é a **Arquitetura em Camadas**, complementada pelo estilo **Model-View-Template (MVT)** nativo do Django na fronteira web. As camadas, do topo (mais próximo do usuário) à base (infraestrutura), são: **Apresentação, Interface/Borda HTTP, Aplicação/Serviços, Domínio e Persistência/Infraestrutura**. A regra de dependência é unidirecional: camadas superiores conhecem as inferiores por meio de abstrações, e a camada de domínio não depende de nenhuma outra.

2.3. Justificativa da escolha

A arquitetura em camadas foi escolhida por quatro motivos alinhados ao contexto do projeto:

- **Testabilidade:** como as regras de negócio dependem de uma interface de repositório, é possível testá-las com uma implementação em memória, sem banco de dados, o que sustenta a suíte de testes automatizados da equipe.
- **Adaptação ao ambiente de hospedagem:** o projeto é demonstrado no plano gratuito do Render sem banco de dados persistente. A troca transparente entre persistência em memória e PostgreSQL, sem alterar a lógica de negócio, atende diretamente a essa restrição.

- **Manutenibilidade e evolução:** a separação por responsabilidade permite adicionar novas regras ou novos meios de persistência com impacto localizado, atendendo ao requisito não funcional de evolução futura.
- **Baixo acoplamento ao framework:** entidades e serviços não importam tipos do Django em sua essência, reduzindo o risco de dependência excessiva de uma tecnologia específica.

3. DIAGRAMA DE ARQUITETURA

3.1. Diagrama de componentes em camadas

Figura 1 — Organização em camadas da aplicação e sentido das dependências.



3.2. Descrição dos componentes

Componente	Responsabilidade
web_ui/	Camada de apresentação: templates Django estilizados com Tailwind CSS e o cliente JavaScript que consome a API. Não contém regra de negócio.
views.py	Controladores finos: convertem a requisição HTTP em chamadas aos serviços e formatam a resposta. Não implementam regra de negócio.
http.py	Borda HTTP: decorador <code>api_view</code> (validação de método, autenticação por token Bearer e tradução de erros de domínio em respostas JSON) e utilitários de payload/resposta.
services/	Camada de aplicação: contém as regras de negócio (cadastro, autenticação, candidatura, perfil, gestão de vagas), sempre recebendo o repositório como dependência.
domain/	Entidades de domínio (dataclasses imutáveis/tipadas) e a hierarquia de erros de domínio. Núcleo independente de framework e de banco.

Componente	Responsabilidade
repositories/	Contrato de repositório (interface) e suas implementações: memória e Django ORM. O provider seleciona a implementação em tempo de configuração.
config/	Configuração do projeto: seleção do repositório, segurança (HTTPS/HSTS/CSRF), arquivos estáticos (WhiteNoise) e roteamento raiz.

3.3. Fluxo de uma requisição típica

O fluxo de uma candidatura a uma vaga ilustra a colaboração entre as camadas:

- **1.** O cliente envia uma requisição HTTP autenticada para a rota `/api/v1/internships/<slug>/applications/`.
- **2.** O decorador `api_view` (`http.py`) valida o método, resolve o token Bearer e obtém o `user_id`.
- **3.** A view (`views.py`) delega a operação ao serviço `apply()` em `services/platform.py`.
- **4.** O serviço aplica as regras de negócio (vaga ativa, prazo, curso/semestre elegível, ausência de candidatura duplicada) usando o repositório injetado.
- **5.** O repositório concreto (memória ou Django) persiste a candidatura e devolve uma entidade de domínio.
- **6.** Erros de domínio são convertidos automaticamente em respostas JSON com o código HTTP adequado pela borda HTTP.

4. DECISÕES DE DESIGN E PRINCÍPIOS SOLID

O projeto aplica princípios SOLID de forma concreta. Destacam-se a seguir, com evidência no código, os quatro princípios mais relevantes para a arquitetura adotada (atendendo e superando o mínimo de dois princípios exigido).

4.1. DIP — Princípio da Inversão de Dependência

As regras de negócio não dependem de uma implementação concreta de persistência, e sim de uma **abstração**: o *Protocol* PlatformRepository, definido em repositories/contracts.py. Toda função de serviço recebe esse contrato como parâmetro, invertendo a dependência da política (negócio) em relação ao detalhe (banco de dados).

```
# platform_api/repositories/contracts.py
class PlatformRepository(Protocol):
    generation: str
    def get_user(self, user_id: int) -> User | None: ...
    def has_application(self, user_id: int, slug: str) -> bool: ...
    def create_application(self, user_id, slug, cover_letter) -> Application: ...
    # ... demais operações do domínio

# platform_api/services/platform.py
def apply(repo: PlatformRepository, user_id: int, slug: str, data: dict) -> dict:
    item = repo.get_internship(slug)          # depende da ABSTRAÇÃO, não do Django
    ...
    return repo.create_application(user_id, slug, cover_letter).as_dict()
```

4.2. OCP — Princípio do Aberto/Fechado

O sistema está **aberto para extensão e fechado para modificação**: adicionar um novo mecanismo de persistência significa criar uma nova classe que satisfaça o contrato PlatformRepository, **sem alterar** nenhuma função de serviço já existente. Foi exatamente assim que as implementações InMemoryRepository e DjangoRepository coexistem.

4.3. LSP — Princípio da Substituição de Liskov

As implementações InMemoryRepository e DjangoRepository são **substituíveis entre si**: ambas honram a mesma interface e os mesmos contratos de retorno (entidades de domínio), de modo que os serviços operam corretamente com qualquer uma delas. A suíte de testes exercita as regras de negócio sobre a implementação em memória justamente por essa substituíbilidade.

4.4. SRP — Princípio da Responsabilidade Única

Cada módulo possui uma única razão para mudar: domain/entities.py representa dados; services/ concentra regras de negócio; repositories/ trata de persistência; http.py e views.py cuidam da fronteira HTTP. Por exemplo, a função register() em

`services/auth.py` tem a responsabilidade exclusiva de validar e criar um usuário, delegando a persistência ao repositório e a formatação da resposta às entidades.

5. PADRÕES DE PROJETO (DESIGN PATTERNS)

Foram empregados padrões de projeto consolidados. A seguir, os principais, com sua motivação e evidência no código (atendendo e superando o mínimo de um padrão exigido).

5.1. Repository (Repositório)

O padrão **Repository** isola a lógica de acesso a dados atrás de uma coleção orientada ao domínio. O contrato `PlatformRepository` define as operações; `InMemoryRepository` e `DjangoRepository` as implementam. Com isso, os serviços manipulam entidades de domínio sem conhecer detalhes de SQL, ORM ou estruturas em memória.

5.2. Provider / Factory (Fábrica)

A criação do repositório concreto é centralizada em uma **fábrica**, `get_repository()` em `repositories/provider.py`, que decide qual implementação instanciar conforme a configuração `settings.PLATFORM_REPOSITORY`. O restante do sistema importa apenas o objeto `repository` já resolvido, sem conhecer a regra de seleção.

```
# platform_api/repositories/provider.py
def get_repository():
    if settings.PLATFORM_REPOSITORY == "django":
        from platform_api.repositories.django import repository
    else:
        from platform_api.repositories.memory import repository
    return repository

repository = get_repository()
```

5.3. Strategy (Estratégia)

A persistência funciona como uma **estratégia intercambiável**: memória e Django são duas estratégias que cumprem o mesmo contrato, selecionadas por configuração (a variável de ambiente `PLATFORM_REPOSITORY`, que assume "django" quando há `DATABASE_URL` e "memory" caso contrário). Trocar a estratégia não exige qualquer alteração nas regras de negócio.

5.4. Decorator (Decorador)

O decorador `api_view` (`http.py`) acrescenta comportamentos transversais às views — validação de método HTTP, exigência de autenticação e conversão de exceções de

domínio em respostas JSON — **sem duplicar código** em cada endpoint. Trata-se de uma aplicação idiomática do padrão Decorator em Python.

```
# platform_api/http.py (trecho)
def api_view(methods, *, authenticated=False, company=False):
    def decorator(view):
        @wraps(view)
        def wrapped(request, *args, **kwargs):
            if request.method not in allowed: ...          # valida método
            try:
                if authenticated:
                    kwargs['user_id'] = authenticated_user_id(request)
                    return view(request, *args, **kwargs)
            except DomainError as exc:                    # traduz erro -> JSON
                return JsonResponse({'error': {...}}, status=exc.status)
            return wrapped
        return decorator
```

6. TECNOLOGIAS UTILIZADAS E JUSTIFICATIVAS

Tecnologia	Papel	Justificativa
Python 3.13	Linguagem	Sintaxe expressiva, tipagem opcional (type hints) e ampla adoção acadêmica, favorecendo legibilidade e manutenção.
Django 5.2	Framework web	Framework maduro com ORM, autenticação, segurança (CSRF, XSS, HSTS) e roteamento integrados, acelerando o desenvolvimento seguro do MVP.
Tailwind CSS	Interface	Estilização utilitária e responsiva (integrada via django-tailwind), atendendo ao requisito de responsividade de 360px a 1920px.
PostgreSQL	Banco (futuro)	SGBD relacional robusto previsto para a persistência definitiva (Supabase), integrado por dj-database-url via DATABASE_URL.
Gunicorn	Servidor WSGI	Servidor de aplicação WSGI para produção, executando a aplicação Django no ambiente de hospedagem.
WhiteNoise	Estáticos	Serve arquivos estáticos comprimidos diretamente pela aplicação, dispensando servidor de estáticos dedicado.
Render	Hospedagem	Plataforma de deploy com HTTPS e health check, adequada à demonstração pública gratuita do sistema.

7. ATRIBUTOS DE QUALIDADE SUPORTADOS PELA ARQUITETURA

As decisões arquiteturais sustentam diretamente requisitos não funcionais do projeto:

- **Segurança:** HTTPS obrigatório, redirecionamento SSL, HSTS, proteção CSRF/XSS do Django, hashing de senhas e tokens assinados por HMAC (`django.core.signing`).
- **Manutenibilidade:** separação em camadas e baixo acoplamento ao framework facilitam correções e evolução.
- **Portabilidade/Flexibilidade:** a estratégia de persistência intercambiável permite operar com ou sem banco de dados.
- **Confiabilidade:** o repositório em memória protege a consistência das operações durante a sessão por meio de exclusão mútua (`RLock`).

8. CONSIDERAÇÕES FINAIS

A arquitetura em camadas, associada à inversão de dependência e aos padrões Repository, Provider/Factory, Strategy e Decorator, confere ao sistema clareza estrutural, testabilidade e capacidade de evolução. As decisões documentadas não são meramente teóricas: cada uma corresponde a um trecho verificável do código-fonte. Como próximos passos naturais, destacam-se a conclusão da integração com o PostgreSQL (Supabase) por meio da estratégia de persistência já preparada e a ampliação da cobertura de testes de integração sobre a implementação baseada em ORM.